Progetto di Guida Autonoma con Behavioral Cloning

Implementazione di un Agente Intelligente per TORCS utilizzando k-Nearest Neighbors

Corso di Intelligenza Artificiale: Metodi ed Applicazioni
Gruppo 23
A.A. 2024-2025

Indice

1 Introduzione **2**
1.1 Obiettivi del Progetto 2

2 Background Teorico **2**
2.1 TORCS - The Open Racing Car Simulator 2
2.2 Behavioral Cloning 2
2.3 k-Nearest Neighbors (k-NN) 3

3 Architettura del Sistema **3**
3.1 Architettura TORCS 3
3.2 Componenti del Sistema 3

4 Implementazione **3**
4.1 Spazio degli Stati 3
4.2 Spazio delle Azioni 4
4.3 Algoritmo k-NN 4
4.4 Codice Principale - SimpleDriver 4
4.5 Raccolta Dati 5

5 Risultati Sperimentali **5**
5.1 Dataset di Training 5
5.2 Prestazioni del Sistema 6
5.3 Gestione delle Situazioni Critiche 6

6 Discussione **6**
6.1 Vantaggi dell'Approccio 6
6.2 Limitazioni 6

7 Conclusioni **7**

1 Introduzione

Il presente documento descrive lo sviluppo e l'implementazione di un sistema di guida autonoma per il simulatore TORCS (The Open Racing Car Simulator) utilizzando tecniche di *Behavioral Cloning* e classificazione tramite algoritmo k-Nearest Neighbors (k-NN).

Il progetto si inserisce nel contesto dell'intelligenza artificiale applicata alla guida autonoma, un campo di ricerca di crescente importanza che trova applicazioni in ambiti che vanno dalla robotica ai veicoli autonomi reali.

1.1 Obiettivi del Progetto

Gli obiettivi principali del progetto sono:

- Implementare un agente intelligente capace di guidare autonomamente in TORCS
- Utilizzare l'approccio del *Behavioral Cloning* per l'apprendimento del comportamento di guida
- Applicare l'algoritmo k-NN per la classificazione delle azioni di guida
- Sviluppare un sistema di raccolta dati automatizzato per la costruzione del dataset di training

2 Background Teorico

2.1 TORCS - The Open Racing Car Simulator

TORCS è un simulatore open source di corse automobilistiche sviluppato nel 1997, inizialmente concepito come videogioco ma successivamente adottato dalla comunità scientifica come piattaforma di riferimento per competizioni basate su algoritmi di intelligenza artificiale per la guida autonoma.

Il simulatore implementa un ambiente tridimensionale completo che include:

- Fisica realistica del veicolo (motore, consumo carburante, frizione, danni)
- Modellazione aerodinamica
- Sensori di telemetria completi
- Architettura client-server per l'integrazione di agenti esterni

2.2 Behavioral Cloning

Il *Behavioral Cloning* è una tecnica di apprendimento automatico che consiste nell'addestrare un modello a imitare il comportamento di un esperto umano. Nel contesto della guida autonoma, questa tecnica prevede:

1. **Raccolta dati**: Registrazione delle azioni compiute da un guidatore umano esperto
2. **Preprocessing**: Elaborazione e normalizzazione dei dati sensoriali
3. **Training**: Addestramento del modello sui dati raccolti

2.3 k-Nearest Neighbors (k-NN)

L'algoritmo k-NN è un metodo di classificazione *lazy learning* che classifica un nuovo campione basandosi sulla classe maggioritaria dei suoi k vicini più prossimi nello spazio delle caratteristiche. La sua semplicità e efficacia lo rendono particolarmente adatto per problemi di classificazione in tempo reale.

3 Architettura del Sistema

3.1 Architettura TORCS

Il sistema TORCS utilizza un'architettura client-server basata su protocollo UDP:

- **Server**: Motore di gioco TORCS con server bot dedicati
- **Client**: Agente Java che implementa la logica di guida
- **Comunicazione**: Scambio di messaggi contenenti dati sensoriali e comandi di controllo

3.2 Componenti del Sistema

Il sistema implementato è composto dai seguenti moduli principali:

1. **SimpleDriver**: Classe principale che implementa la logica di guida
2. **KNearestNeighbor**: Implementazione dell'algoritmo k-NN
3. **KDTree**: Struttura dati per ricerca efficiente dei vicini
4. **Sample**: Classe per la rappresentazione dei campioni di training
5. **Moduli di comunicazione**: Client, SocketHandler, MessageParser

4 Implementazione

4.1 Spazio degli Stati

Il sistema utilizza le seguenti caratteristiche sensoriali come input:

Sensore	**Range**	**Descrizione**
angle	$[-\pi, +\pi]$	Angolo tra asse longitudinale e tracciato
speedX	$(-\infty, +\infty)$	Velocità longitudinale
speedY	$(-\infty, +\infty)$	Velocità laterale
track[0..18]	$[0, 200]$	Distanza dai bordi della pista
trackPos	$(-\infty, +\infty)$	Posizione relativa sull'asse della pista

Tabella 1: Principali sensori utilizzati

4.2 Spazio delle Azioni

Il sistema è stato progettato per classificare le situazioni di guida in 10 classi distinte:

Classe	Azione
0	Fermata (nessuna accelerazione, nessuna sterzata)
1	Accelerazione in rettilineo
2	Accelerazione con sterzata a sinistra
3	Accelerazione con sterzata a destra
4	Sterzata a sinistra senza accelerazione
5	Frenata
6	Sterzata a destra senza accelerazione
7	Retromarcia con sterzata a destra
8	Retromarcia
9	Retromarcia con sterzata a sinistra

Tabella 2: Classi di azioni implementate

4.3 Algoritmo k-NN

L'implementazione dell'algoritmo k-NN utilizza una struttura dati KD-Tree per ottimizzare la ricerca dei vicini più prossimi. Le caratteristiche principali includono:

- **Distanza euclidea** per il calcolo della similarità
- **Valore k dispari** per evitare situazioni di parità
- **Ricerca efficiente** tramite KD-Tree con complessità $O(\log n)$

4.4 Codice Principale - SimpleDriver

La classe `SimpleDriver` rappresenta il cuore del sistema. Di seguito sono riportate le sezioni più significative:

Listing 1: Costruttore della classe SimpleDriver

```
public SimpleDriver(int numberOfNeighbors) {
    if((numberOfNeighbors%2 != 0) && (numberOfNeighbors > 0)){
        this.knnFinder = new KNearestNeighbor("training.csv");
        this.numberOfNeighbors = numberOfNeighbors;
    } else {
        System.out.println("Errore nella scelta del numero di
           vicini");
        System.exit(1);
    }
}
```

Listing 2: Metodo di controllo principale

```
public Action control(SensorModel sensors) {
    Action action = new Action();
    action.gear = getGear(sensors);
```

```
4
5       // Creazione del vettore delle caratteristiche
6       double[] features = new double[]{
7           sensors.getAngleToTrackAxis(),
8           sensors.getSpeed(),
9           sensors.getLateralSpeed(),
10          // ... sensori track[0..18] ...
11          sensors.getTrackPosition()
12      };
13
14      Sample attuale = new Sample(features);
15      int cls = knnFinder.classify(attuale, numberOfNeighbors);
16
17      // Esecuzione azione basata sulla classe predetta
18      switch (cls) {
19          case 0: /* Fermata */ break;
20          case 1: /* Accelerazione */ break;
21          // ... altre classi ...
22      }
23
24      collectDataForCSV(features, action, cls);
25      return action;
26  }
```

4.5 Raccolta Dati

Il sistema implementa un meccanismo automatizzato per la raccolta dei dati di training:

Listing 3: Raccolta dati per training

```
1  private void collectDataForCSV(double[] features, Action action,
       int classe) {
2       DataEntry entry = new DataEntry(
3           features[0], features[1], features[2],
4           Arrays.copyOfRange(features, 3, 22),
5           features[22], classe
6       );
7       dataEntries.add(entry);
8   }
```

5 Risultati Sperimentali

5.1 Dataset di Training

Il dataset di training è stato costruito attraverso sessioni di guida manuale, registrando:

- Configurazioni sensoriali
- Azioni compiute dal guidatore
- Classificazione delle situazioni di guida

5.2 Prestazioni del Sistema

Il sistema implementato dimostra le seguenti caratteristiche:

- **Tempo di risposta**: Classificazione in tempo reale (¡ 10ms per decisione)
- **Stabilità**: Comportamento consistente in situazioni simili
- **Adattabilità**: Capacità di gestire diverse configurazioni di tracciato

5.3 Gestione delle Situazioni Critiche

Il sistema gestisce diverse situazioni critiche:

- **Uscita di pista**: Manovre di recupero tramite retromarcia
- **Curve strette**: Riduzione velocità e sterzata progressiva
- **Ostacoli**: Frenata d'emergenza quando necessario

6 Discussione

6.1 Vantaggi dell'Approccio

L'approccio implementato presenta diversi vantaggi:

- **Semplicità**: k-NN non richiede training offline complesso
- **Interpretabilità**: Le decisioni sono facilmente comprensibili
- **Robustezza**: Gestione efficace di situazioni non viste durante il training
- **Efficienza**: Risposta in tempo reale grazie al KD-Tree

6.2 Limitazioni

Sono state identificate le seguenti limitazioni:

- **Dipendenza dai dati**: Qualità delle prestazioni legata al dataset di training
- **Memoria**: Necessità di mantenere tutto il dataset in memoria
- **Scalabilità**: Prestazioni degradano con dataset molto grandi

7 Conclusioni

Il progetto ha dimostrato la fattibilità dell'approccio di *Behavioral Cloning* utilizzando k-NN per la guida autonoma in TORCS. Il sistema implementato è in grado di guidare autonomamente il veicolo simulato, imitando il comportamento di un guidatore umano esperto.

L'utilizzo dell'algoritmo k-NN si è rivelato efficace per la classificazione in tempo reale delle situazioni di guida, fornendo un buon compromesso tra semplicità implementativa e performance operative.

Il sistema di raccolta dati automatizzato consente un miglioramento continuo delle prestazioni attraverso l'espansione del dataset di training, rendendo l'approccio scalabile e adattabile a diversi scenari di guida.